

# COMPLEJIDAD ALGORITMICA

---

**PROGRAMACION III**

**INF - 131**

**Lic. Marcelo Aruquipa**

# Introducción

---

- Uno de los objetivos de desarrollar un programa es:

“UTILIZAR MENOS RECURSOS (TIEMPO Y MEMORIA)  
DE LA COMPUTADORA”

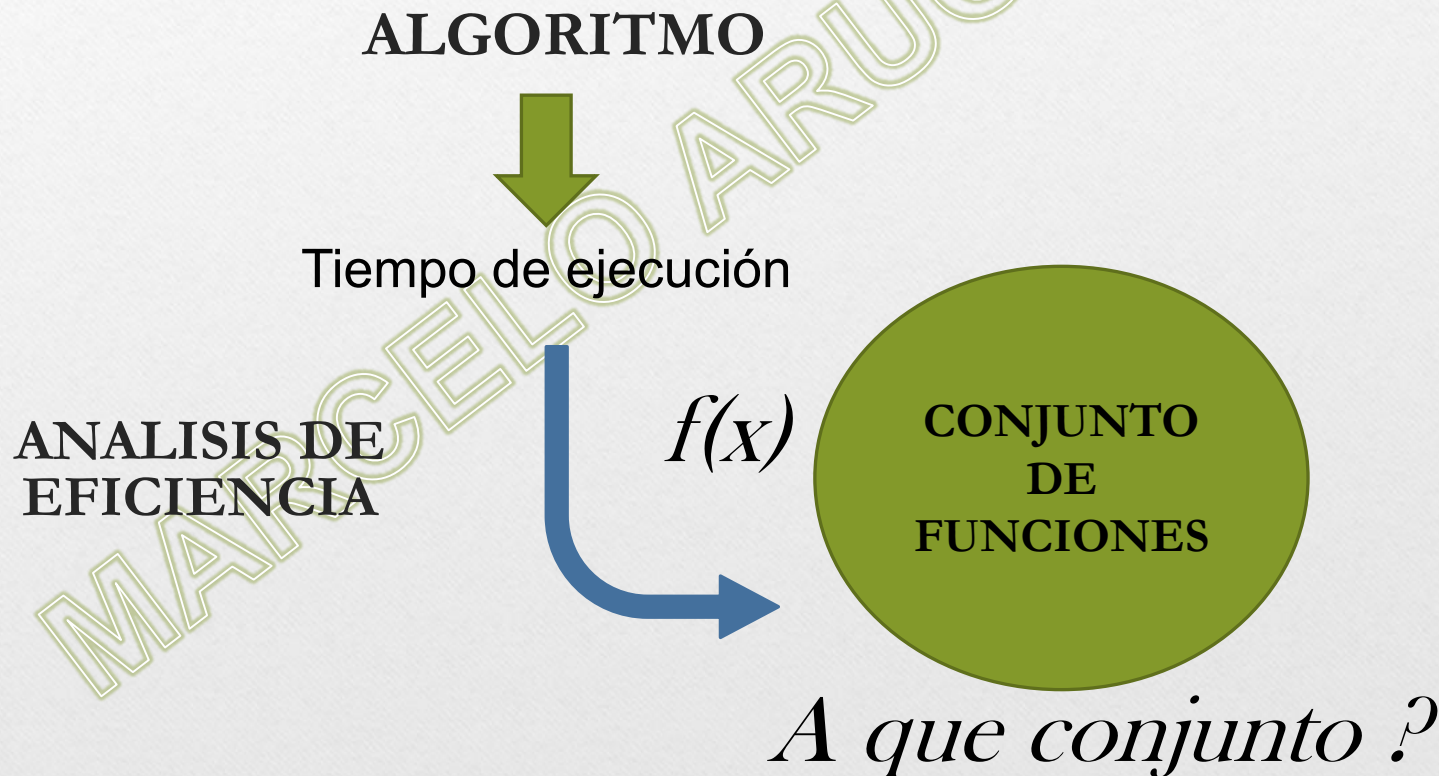
Para alcanzar la solución.

- A esto se denomina:

*“ALGORITMO EFICIENTE”*

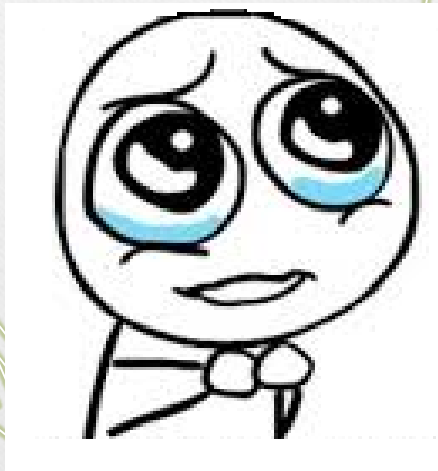


# Eficiencia de un Algoritmo



# TENGO MI ALGORITMO YA PUEDO CODIFICARLO?

---





# Análisis de Eficiencia

“Factor decisivo para el diseño del algoritmo”

A Priori  
(Teórico)

Buscar función que  
represente al tiempo  
de ejecución

A Posteriori  
(Experimental)

Calculo de tiempo en  
forma práctica.  
Estadísticas

# Tamaño del problema

---

- El tamaño de la entrada(numero de datos) de un problema permitirá medir la complejidad de un algoritmo
- Uso de recursos → tamaño de la entrada
  - Nro de datos del problema
  - Nro de dígitos de numero
  - Nro de elementos de un array
  - Nro de caracteres de una cadena
  - Nro de llamadas a una función, recursos,etc.



# Tiempo de Ejecución



Tiempo de Ejecución - Operaciones que realiza

$$T(n): \mathbb{N} \rightarrow \mathbb{R}$$

- Función para un tamaño de entrada  $n$

$$T(n)_{min} \leq T(n) \leq T(n)_{max}$$

# Complejidad Asintótica

- Cada  $T(n)$  de un algoritmo tiene un comportamiento asintótico  $\rightarrow$  agrupar en familias de funciones (ordenes)

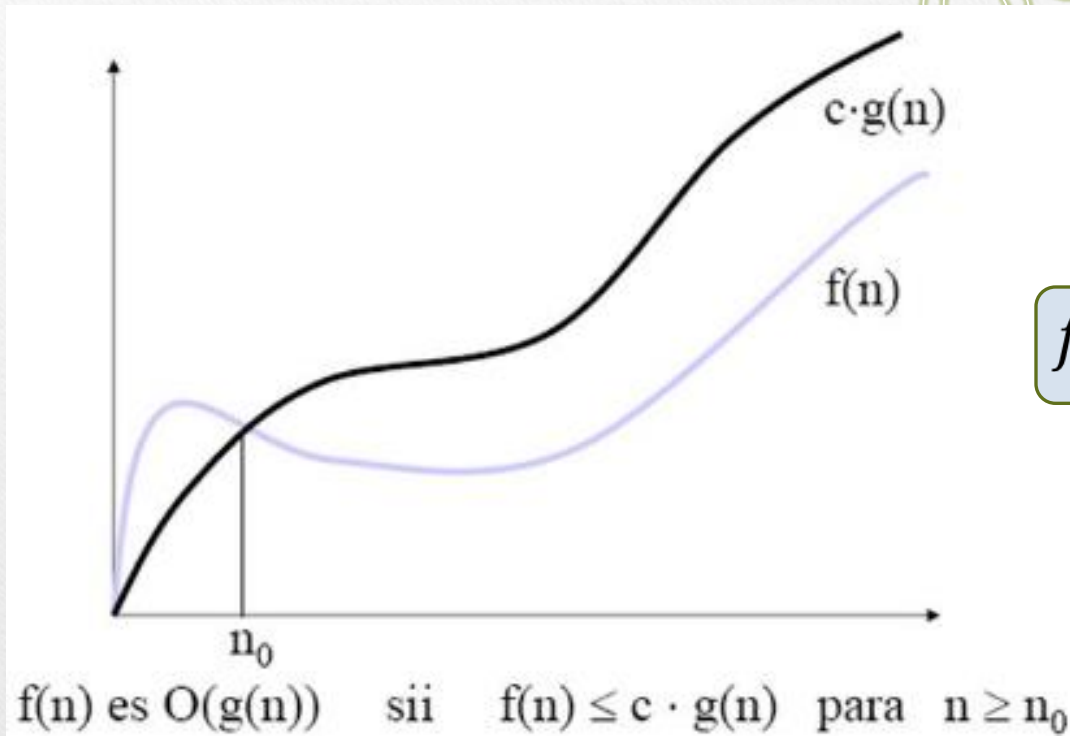
*Comportamiento = Orden de Complejidad*

- Cota Superior  $\mathcal{O}()$

$$\mathcal{O}(g(n)) = \{f(n) / \exists c > 0, n_0 > 0 : f(n) \leq cg(n); \forall n \geq n_0\}$$

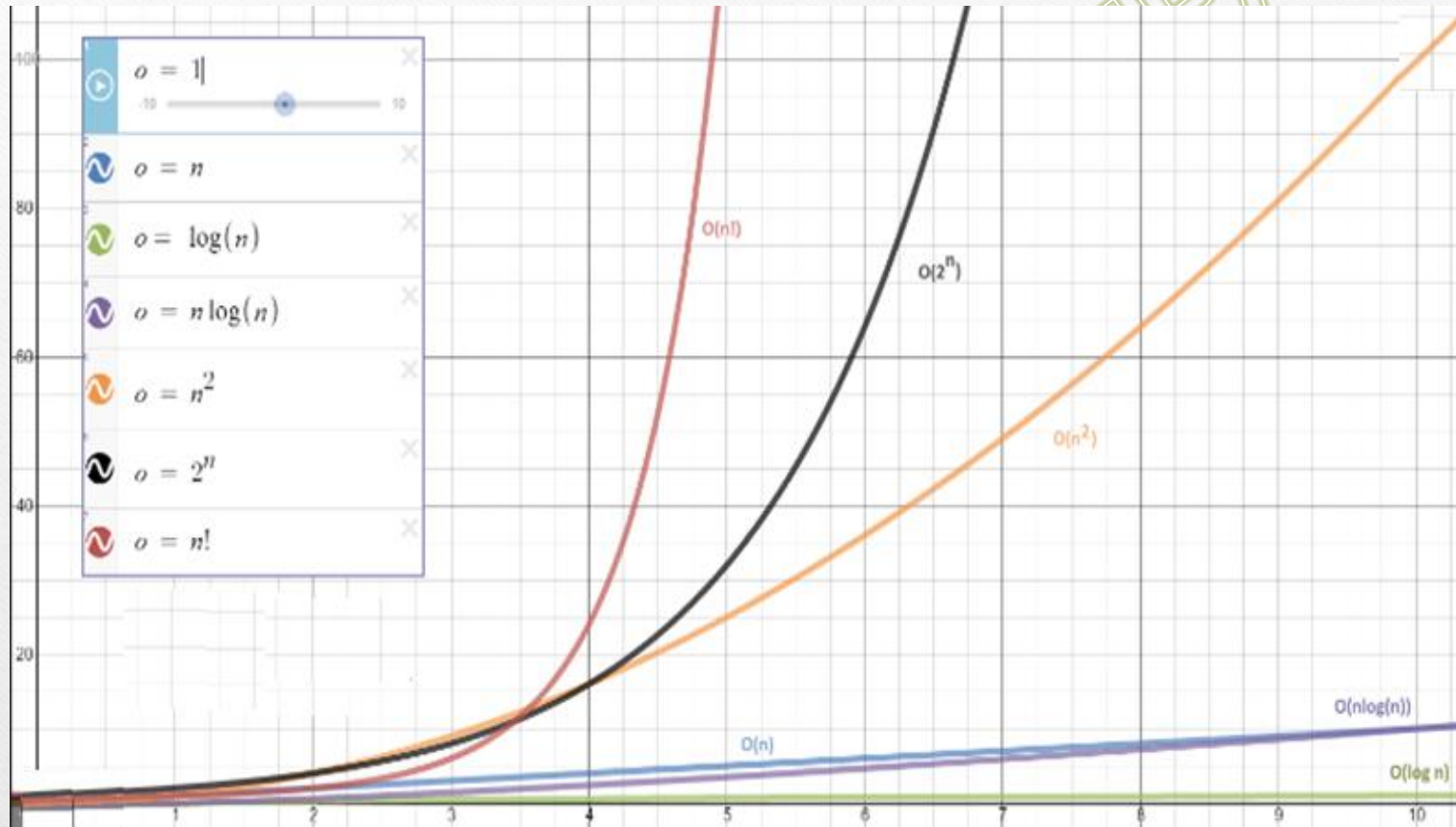


# Complejidad Asintótica



$$f(n) \in O(g(n))$$

# Ordenes de Complejidad





# Ordenes de Complejidad

Eficiente

- $O(1)$  Constante: No depende del tamaño de la entrada
- $O(\log n)$  Logarítmica: Alg. iteración, búsqueda Binaria
- $O(n)$  Lineal: Bucles simples con instrucciones sean cttas
- $O(n \log n)$  Cuasi lineal: alg. tipo divide y vencerás, ordenación

Tratable

- $O(n^2)$  Cuadrática: Bucles anidados dobles, bubble sort
- $O(n^3)$  Cubica: Bucle triple anidación, multiplicación matrices
- $O(n^k)$  Polinomial  $k > 3$ : anidación de bucles

Intratable

- $O(k^n)$  Exponencial  $k > 1$ : Algoritmos recursivos
- $O(n!)$  Factorial: Algoritmos de grafos

$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n)$

# Ordenes de Complejidad

**Ejemplo.**

Sea  $f(n) = 3n^3 + 2n^2$  pertenece a  $\mathcal{O}(n^3)$  ?

**Sol.**

$$3n^3 + 2n^2 \leq cn^3$$

$$3n^3 - cn^3 + 2n^2 \leq 0$$

$$n^3(c - 3) \geq 2n^2$$

$$n^3 \geq \frac{2n^2}{c - 3}; \text{ con } c = 4 \text{ y } n_0 = 2$$

$$\Rightarrow 3n^3 + 2n^2 \in \mathcal{O}(n^3) \text{ si } c = 4 \text{ y } n_0 = 2$$



# Reglas para calcular $T(n)$

- Considerando el peor tiempo posible
  - a) Sentencias sencillas  $S$ ;  $O(1)$
  - b) Secuencia de instrucciones  $T(S_1; S_2) = T(S_1) + T(S_2)$
  - c) Decision: la condición sera  $O(1)$  + la rama Then o Else que mas tiempo se ejecuta
  - d) Bucles:
    - a) Bucle con evolución lineal,  $n$  como limite

```
for (int i = 0; i < n; i++) {  
    O(1);  
}
```

$$N * O(1) = O(n)$$

# Reglas para calcular $T(n)$

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        O(1);  
    }  
}
```

$$N * N * O(1) = O(n^2)$$

## b) Bucle con evolución no lineal

```
int c = 1;  
while(c < n){  
    O(1);  
    c = 2*c;  
}
```

```
int c = n;  
while(c > 0){  
    O(1);  
    c = c/2;  
}
```

$$2^k \geq N \rightarrow k = \log_2(n) \text{ es } O(\log n)$$



# Reglas para calcular $T(n)$

```
for (int i = 0; i < n; i++) {  
    int c = i;  
    while(c > 1){  
        O(1);  
        c = c/2;  
    }  
}
```

$$N * O(\log n) = O(n \log n)$$

# Ejemplo 1

Suponiendo  $n$  como tamaño de la entrada:

| Nro Linea                    | Codigo                                                    | Frecuencia |
|------------------------------|-----------------------------------------------------------|------------|
| 1                            | <code>j = 1;</code>                                       | 1          |
| 2                            | <code>while((A[j] &lt; c) &amp;&amp; (j &lt; n)) {</code> | $n$        |
| 3                            | <code>    j++;</code>                                     | $n - 1$    |
|                              | <code>}</code>                                            |            |
| 4                            | <code>if(A[j] == c)</code>                                | 1          |
| 5                            | <code>    return j;</code>                                | 1          |
|                              | <code>else</code>                                         |            |
| 6                            | <code>    return 0;</code>                                |            |
|                              | $T(n) =$                                                  | $2n + 2$   |
| Por lo tanto $T(n) \in O(n)$ |                                                           |            |



# Ejemplo 2

Suponiendo  $n$  como tamaño de la entrada:

| Nro Linea                           | Codigo                        | Frecuencia           |
|-------------------------------------|-------------------------------|----------------------|
| 1                                   | for (int i = 0; i < n; i++) { | $n + 1$              |
| 2                                   | c = i;                        | $n$                  |
| 3                                   | while (c > 0) {               | $\log n + 1$         |
| 4                                   | r++;                          | $\log n$             |
| 5                                   | c = c/2;                      | $\log n$             |
|                                     | }                             | $n$                  |
|                                     | }                             |                      |
|                                     | $T(n) =$                      | $3n \log n + 3n + 1$ |
| Por lo tanto $T(n) \in O(n \log n)$ |                               |                      |

# Ejemplo 3

Suponiendo n como tamaño de la entrada:

| Nro Linea | Código                        | Frecuencia                            |
|-----------|-------------------------------|---------------------------------------|
| 1         | k = 0;                        | 1                                     |
| 2         | for(i = 1; i <= n; i++) {     | n+1                                   |
| 3         | for(j = 1; j <= n*n; j++) {   | n <sup>2</sup> +1    n                |
| 4         | for(p = 1; p <= n*n*n; p++) { | n <sup>3</sup> +1    n <sup>2</sup> n |
| 5         | k++;                          | n <sup>3</sup> n <sup>2</sup> n       |
|           | }                             |                                       |
|           | }                             |                                       |
|           | }                             |                                       |

$$T(n) = 1 + (n+1) + n(n^2 + 1) + n(n^2)(n^3+1) + n(n^2)(n^3)$$

Por lo tanto

$$T(n) = 2n^6 + 2n^2 + 2n + 2 \in O(n^k); \quad k > 3 \quad \text{Complejidad polinomial}$$



GRACIAS

---